

Lab 4: VibeCoding

Selected AI Chat History and Development Evidence

Project	17_wordle - Python Wordle game
Repository	https://github.com/vinayvarmavetukuri06/17_wordle
Scope	Only the essential AI prompts, implementation decisions, tests, and Git evidence
Tasks	Task 1: duplicate letters Task 2: turns/win/loss Task 3: 4/5/6 modes Task 4: history/summary

Lab requirement covered

The handout requires cloning the assigned repo, running the original Python game, recording before and after videos, using AI prompts to fix the code and add README features, keeping separate commits for each task, and submitting the videos, updated code, and exported AI chat history under Lab-4.

Note

This PDF is intentionally selective. It contains the conversation material needed to demonstrate the AI-assisted development process, not the entire chat transcript.

Task 1 - Fix duplicate-letter feedback

AI prompt used

I have cloned the existing Wordle Python project.

Inspect the current files, especially feedback.py, before changing anything.

Task 1 from the README is: implement standard Wordle-style duplicate-letter feedback. Exact matches must be resolved first, and each occurrence in the target can satisfy at most one occurrence in the guess.

Please explain the current bug using target "apple" and guess "altee", propose a minimal fix, give the exact replacement code for feedback.py, and give test cases for repeated letters, exact matches, and absent letters. Do not implement Tasks 2-4 yet.

Key implementation

```
def evaluate(target, guess):
    result = ["gray"] * len(guess)
    remaining = list(target)

    # First pass: exact matches.
    for i, ch in enumerate(guess):
        if ch == target[i]:
            result[i] = "green"
            remaining[i] = None

    # Second pass: misplaced letters.
    for i, ch in enumerate(guess):
        if result[i] == "green":
            continue
        if ch in remaining:
            result[i] = "yellow"
            remaining[remaining.index(ch)] = None

    return result
```

Verification captured during the work

Target: apple
Guess: altee
Correct result: ['green', 'yellow', 'gray', 'gray', 'green']

Exact match test: ['green', 'green', 'green', 'green', 'green']
Absent letters test: ['gray', 'gray', 'gray', 'gray', 'gray']

Git evidence

4baa35b Fix duplicate-letter feedback

Task 1 was then reconciled with an existing remote commit using a rebase conflict resolution, and the resulting commit was pushed successfully.

Task 2 - Valid turns, win and loss handling

AI prompt used

I have completed Task 1 in the existing Wordle Python project.

Now implement ONLY Task 2 from the repository README: clean win handling, clean loss handling, exactly 6 accepted guesses, and invalid guesses must NOT consume a turn.

Inspect main.py and game.py first. Preserve the existing project structure, q-to-quit behavior, in-memory design, and do not implement Tasks 3 or 4.

Key implementation change

```
accepted_guesses = 0

while accepted_guesses < 6:
    guess = input("> ").strip().lower()

    if guess == "q":
        print("Game quit.")
        return

    if len(guess) != self.length or not guess.isalpha():
        print("Enter a valid word of the required length.")
        continue

    accepted_guesses += 1
```

Testing performed

Invalid inputs such as **abc** and **abc12** were rejected without consuming a valid turn. The implementation was also tested for early win, sixth-guess win, six-guess loss, and q-to-quit.

Git evidence

e2a804b Handle wins losses and valid turns

Task 3 - 4-, 5-, and 6-letter modes

AI prompt used

I have completed Tasks 1 and 2 in the existing Wordle Python project.

Now implement ONLY Task 3 from the README. Add 4-, 5-, and 6-letter modes using the existing in-memory design. The current words.py has only 5-letter words, so explicitly address how 4- and 6-letter modes can be functional.

Preserve the six accepted-guess limit, invalid-input handling, q-to-quit, duplicate-letter feedback, and do not implement Task 4 or add persistence.

Repository issue identified

The original in-memory word list contained only 5-letter words. To make the required 4- and 6-letter modes actually selectable, the word list was extended in the same in-memory format, while main.py was changed to let the user select 4, 5, or 6.

Final in-memory word-list shape

```
# 4-letter words
"game", "code", "play", "word", "rain"

# 5-letter words
"apple", "array", "eerie", "crane", "grape",
"stare", "steam", "stone", "plant", "flame"

# 6-letter words
"planet", "python", "garden", "friend", "rocket"
```

Mode selection added to main.py

```
print("Choose word length: 4, 5, or 6.")

while True:
    choice = input("> ").strip().lower()
    if choice == "q":
        print("Game quit.")
        break
    if choice in {"4", "5", "6"}:
        WordleGame(length=int(choice)).run()
        break
    print("Enter 4, 5, or 6.")
```

Git evidence

```
c3bb326 Add 4 5 and 6 letter modes
```

Task 4 - Guess history and final summary

AI prompt used

I have completed Tasks 1, 2, and 3 in the existing Wordle Python project.

Now implement ONLY Task 4 from the repository README. Display readable guess history after every accepted guess and provide a final session summary. Invalid guesses must NOT be added to guess history.

Preserve the 6 accepted-guess limit, 4/5/6-letter modes, duplicate-letter feedback, and q-to-quit. Use the existing in-memory history structure and do not add persistence or unrelated features.

Key implementation idea

```
def _print_history(self):
    print("\nGuess history:")
    for number, (guess, feedback) in enumerate(self.history, start=1):
        print(f"{number}. {guess} -> {' '.join(feedback)}")

def _print_summary(self, result):
    print("\nFinal Summary")
    print(f"Result: {result}")
    print(f"Accepted guesses: {len(self.history)}")
    print(f"Word length: {self.length}")
```

Expected user-visible behavior

```
Guess history:
1. crane -> gray yellow gray green gray
2. stare -> yellow gray gray yellow gray
```

```
Final Summary
Result: Won
Accepted guesses: 2
Word length: 5
Target word: apple
```

Git evidence

```
10e949d Add guess history and session summary
```

Final Git and Submission Evidence

Final commit history recorded in the project

```
10e949d Add guess history and session summary
c3bb326 Add 4 5 and 6 letter modes
e2a804b Handle wins losses and valid turns
4baa35b Fix duplicate-letter feedback
625fc23 Update feedback.py
ac16344 Updated scenario title
```

Final repository state

```
HEAD -> main, origin/main
nothing to commit, working tree clean
```

Selected testing checklist

1	Duplicate-letter scoring verified with apple / allee.
2	Exact-match and absent-letter feedback verified.
3	Invalid guesses do not consume turns.
4	Win and loss handling tested with the six-accepted-guess limit.
5	4-, 5-, and 6-letter modes tested.
6	Guess history appears after accepted guesses.
7	Invalid guesses do not create history entries.
8	Final summary reports Won, Lost, or Quit and accepted guess count.
9	Before and after videos were recorded separately for submission.

Submission package

Lab-4 should contain the before video, after video, updated code, and this exported chat-history PDF. The handout also requires separate commits for each task and no PR to the main SETAPESU26 repository.